For Freeway, we restrict the support to be between –10 and +10. All time steps that are *nonterminal* (i.e., those that don't result in a winning or losing state) give a reward of –1 to penalize taking too much time crossing the road. We reward +10 if the chicken successfully crosses the road and –10 if the game is lost (if the chicken doesn't cross the road before the timer runs out). When the chicken gets hit by a car, the game isn't necessarily lost; the chicken just gets pushed down away from the goal.

Our Dist-DQN will take a state, which is a 128-element vector, and will return 3 separate but equal-sized tensors representing the probability distribution over the support for each of the 3 possible actions (UP, DOWN, NO-OP) given the input state. We will use a 51-element support, so the support and probability tensors will be 51 elements.

If our agent begins the game with a randomly initialized Dist-DQN, takes action UP, and receives a reward of –1, how do we update our Dist-DQN? What is the target distribution and how do we compute a loss function between two distributions? Well, we use whatever distribution the Dist-DQN returns for the subsequent state, s_{t+1}, as a prior distribution, and we update the prior distribution with the single observed reward, r_t, such that a little bit of the distribution gets redistributed around the observed r_t.

If we start with a uniform distribution and observe $r_t = -1$, the posterior distribution should no longer be uniform, but it should still be pretty close (figure 7.9). Only if we repeatedly observe $r_t = -1$ for the same state should the distribution start to strongly peak around –1. In normal Q-learning, the discount rate, γ (gamma), controlled how much the expected future rewards contribute to the value of the current state. In distributional Q-learning, the γ parameter controls how much we update the prior toward the observed reward, which achieves a similar function (figure 7.10).

Figure 7.9. We've created a function that takes a discrete distribution and updates it based on observed rewards. This function is performing a kind of approximate Bayesian inference by updating a prior distribution into a posterior distribution. Starting from a uniform distribution (on top, we observe some rewards and we get a peaked distribution at 0 (shown in the middle), and then we observe even more rewards (all zeros), and the distribution becomes a narrow, normal-like distribution (as shown on the bottom).